

EECE 7205: Fundamentals of Computer Engineering

Fall 2026

InClass Work

Topic: Bubble, Insertion, Selection, Quick, Merge, Heap Sort

Student Name	Lingtong Ge
NUID	003917506
Email	ge.lingt@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	9.30

Submission Checklist

- Submit **one PDF** generated from this Overleaf report.
 - Include **clear screenshots** for each problem.
 - File name: `Assignment01_Ge.pdf`
-

Problem 1: Bubble Sort

1. Understanding the Problem

- Goal: sort $A = \{34, 7, 23, 32, 5, 62, 14, 19\}$ ($n = 8$) into increasing order using Bubble Sort.
- Input: an integer array and its size n .
- Output: the array printed after every pass, the sorted array, and the total number of comparisons and swaps.
- Constraints: no library sorting; exactly two main functions, `printArray` and `bubbleSort`; all array printing goes through `printArray`; the sort works in place.
- Edge cases: already sorted (best case), reversed (worst case), duplicates, size 0 or 1.

2. Solution Approach

Bubble Sort walks through the array from left to right and compares each pair of neighbors `arr[j]` and `arr[j+1]`. If the left one is larger, the two are swapped. Doing this across the whole unsorted part is one pass. During a pass the largest remaining value is carried to the right and ends up at the end of the unsorted part. After pass k , the last k positions are final. A **swapped** flag is reset to false at the start of each pass and set to true on any swap. If a full pass makes no swaps, the array is sorted and the loop stops.

3. Pseudocode

```

1 PROCEDURE printArray(arr, n)
2   1. FOR k <- 0 TO n - 1 DO
3     2.   PRINT arr[k] followed by a space
4   3. END FOR
5   4. PRINT newline
6
7 PROCEDURE bubbleSort(arr, n, passes, comparisons, swaps)
8   1. passes <- 0; comparisons <- 0; swaps <- 0
9   2. FOR i <- 0 TO n - 2 DO
10    3.   swapped <- FALSE
11    4.   FOR j <- 0 TO n - 2 - i DO
12      5.     comparisons <- comparisons + 1
13      6.     IF arr[j] > arr[j + 1] THEN
14        7.       SWAP arr[j] AND arr[j + 1]
15        8.       swaps <- swaps + 1
16        9.       swapped <- TRUE
17    10.   END IF
18    11. END FOR
19    12. passes <- passes + 1
20    13. PRINT "Pass", passes, ": "; CALL printArray(arr, n)
21    14. IF swapped = FALSE THEN BREAK
22    15. END FOR

```

4. Hand Trace

Pass 1 (start: 34 7 23 32 5 62 14 19):

j	Comparison	Swap?	Array
0	34 > 7	yes	7 34 23 32 5 62 14 19
1	34 > 23	yes	7 23 34 32 5 62 14 19
2	34 > 32	yes	7 23 32 34 5 62 14 19
3	34 > 5	yes	7 23 32 5 34 62 14 19
4	34 > 62	no	7 23 32 5 34 62 14 19
5	62 > 14	yes	7 23 32 5 34 14 62 19
6	62 > 19	yes	7 23 32 5 34 14 19 62

Pass 1: 7 comparisons, 6 swaps. 62 is in place.

Summary of all passes:

Pass	Comparisons	Swaps	Array after the pass
1	7	6	7 23 32 5 34 14 19 62
2	6	3	7 23 5 32 14 19 34 62
3	5	3	7 5 23 14 19 32 34 62
4	4	3	5 7 14 19 23 32 34 62
5	3	0	5 7 14 19 23 32 34 62
Total	25	15	

5. Correctness

- Invariant: after pass k , the last k elements are the k largest values, in sorted order.
- Termination: after at most $n - 1$ passes, the last $n - 1$ elements are final. If the flag stops the loop earlier, the array is sorted.
- Duplicates: strict $\text{arr}[j] > \text{arr}[j+1]$, so equal values are never swapped. Bubble Sort is stable.
- $n = 0$ or $n = 1$: outer loop does not run.

6. Complexity Analysis

Case	Input	Passes	Comparisons	Time
Best	already sorted	1	$n - 1$	$O(n)$
Average	random order	about n	about $n(n - 1)/2$	$O(n^2)$
Worst	reversed	$n - 1$	$n(n - 1)/2$	$O(n^2)$

7. C++ Program

```
1 // Assignment01_Problem01_Ge.cpp
2 #include <iostream>
3 using namespace std;
4
5 void printArray(const int arr[], int n) {
6     for (int k = 0; k < n; k++) {
7         cout << arr[k];
8         if (k < n - 1) cout << " ";
9     }
10    cout << endl;
11 }
12
13 void bubbleSort(int arr[], int n, int &passes, int &comparisons, int &
    swaps) {
14     passes = 0; comparisons = 0; swaps = 0;
15     for (int i = 0; i < n - 1; i++) {
16         bool swapped = false;
17         for (int j = 0; j < n - 1 - i; j++) {
18             comparisons++;
19             if (arr[j] > arr[j + 1]) {
20                 int temp = arr[j];
21                 arr[j] = arr[j + 1];
22                 arr[j + 1] = temp;
23                 swaps++;
24                 swapped = true;
25             }
26         }
27         passes++;
28         cout << "Pass " << passes << ": ";
29         printArray(arr, n);
```


Input	Passes	Comparisons	Swaps
Original A	5	25	15
Sorted copy	1	7	0

which costs $n - 1$ comparisons. This is the best case, $O(n)$. Without the flag the program would still make all 7 passes and 28 comparisons.

Problem 2: Insertion Sort

1. Understanding the Problem

- Goal: sort A into increasing order using Insertion Sort.
- Output: print the key and the array after each insertion, the sorted array, and total comparisons and shifts.
- Constraints: no library sorting; two main functions; all printing via `printArray`.

2. Solution Approach

Insertion Sort divides the array into a sorted prefix `arr[0..i-1]` and an unsorted suffix. Each step takes `key = arr[i]`, shifts every larger element in the sorted part one position right, and inserts the key in its correct place. The number of shifts equals the number of inversions.

3. Pseudocode

```

1 PROCEDURE printArray(arr, n)
2   1. FOR k <- 0 TO n - 1 DO PRINT arr[k] followed by space
3   2. PRINT newline
4
5 PROCEDURE insertionSort(arr, n, comparisons, shifts)
6   1. comparisons <- 0; shifts <- 0
7   2. FOR i <- 1 TO n - 1 DO
8     3.   key <- arr[i]
9     4.   PRINT "i =", i, ", key =", key, ":"
10    5.   j <- i - 1
11    6.   WHILE j >= 0 DO
12      7.     comparisons <- comparisons + 1
13      8.     IF arr[j] > key THEN
14        9.       arr[j + 1] <- arr[j]
15      10.      shifts <- shifts + 1
16      11.      j <- j - 1
17      12.    ELSE BREAK
18    13.  END WHILE
19    14.  arr[j + 1] <- key
20    15.  CALL printArray(arr, n)
21  16. END FOR

```

4. Hand Trace

i	key	Comparisons	Shifts	Array after the insertion
1	7	1	1	7 34 23 32 5 62 14 19
2	23	2	1	7 23 34 32 5 62 14 19
3	32	2	1	7 23 32 34 5 62 14 19
4	5	4	4	5 7 23 32 34 62 14 19
5	62	1	0	5 7 23 32 34 62 14 19
6	14	5	4	5 7 14 23 32 34 62 19
7	19	6	5	5 7 14 19 23 32 34 62
Total		21	16	

5. Correctness

- Invariant: before the loop, `arr[0..i-1]` is the sorted permutation of the first i elements.
- Termination: i goes from 1 to $n - 1$, each step inserts the key in the correct place.
- Duplicates: strict `arr[j] > key`, so Insertion Sort is stable.
- $n = 0$ or $n = 1$: outer loop does not run.

6. Complexity Analysis

Case	Input	Comparisons	Time
Best	already sorted	$n - 1$	$O(n)$
Average	random	about $n^2/4$	$O(n^2)$
Worst	reversed	$n(n - 1)/2$	$O(n^2)$

7. C++ Program

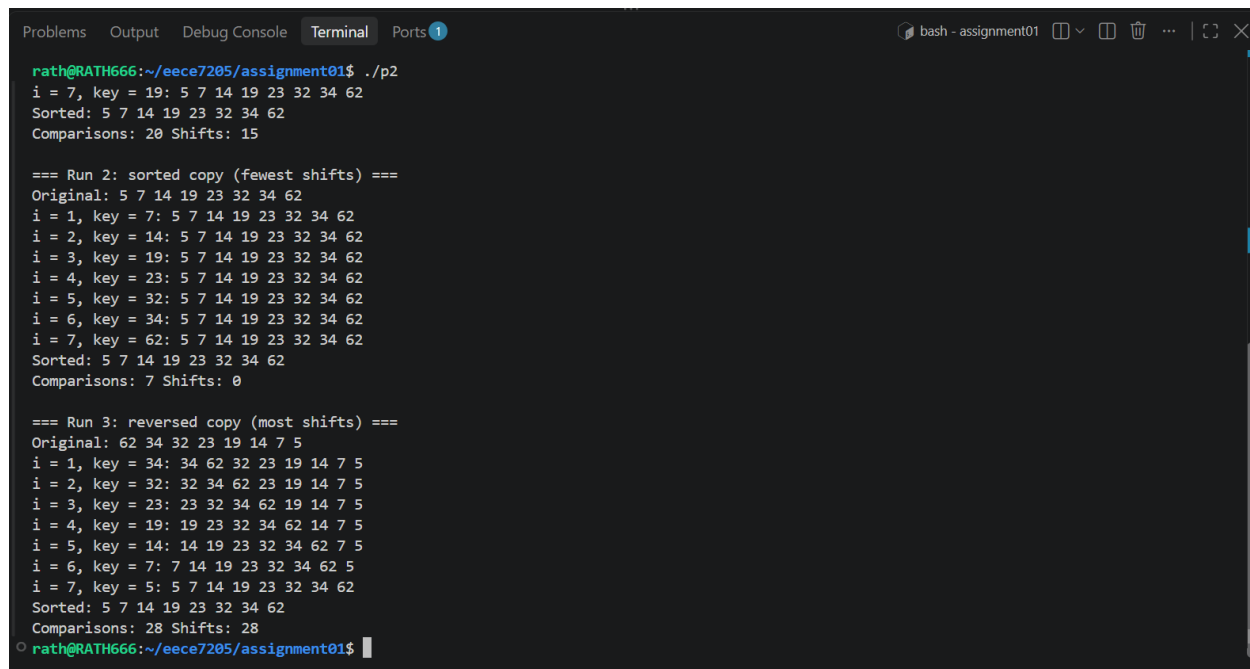
```
1 // Assignment01_Problem02_Ge.cpp
2 #include <iostream>
3 using namespace std;
4
5 void printArray(const int arr[], int n) {
6     for (int k = 0; k < n; k++) {
7         cout << arr[k];
8         if (k < n - 1) cout << " ";
9     }
10    cout << endl;
11 }
12
13 void insertionSort(int arr[], int n, int &comparisons, int &shifts) {
14     comparisons = 0; shifts = 0;
15     for (int i = 1; i < n; i++) {
16         int key = arr[i];
```

```

17     cout << "i = " << i << ", key = " << key << ": ";
18     int j = i - 1;
19     while (j >= 0) {
20         comparisons++;
21         if (arr[j] > key) {
22             arr[j + 1] = arr[j];
23             shifts++;
24             j--;
25         } else break;
26     }
27     arr[j + 1] = key;
28     printArray(arr, n);
29 }
30 }
31
32 int main() {
33     int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
34     int n = sizeof(A) / sizeof(A[0]);
35     int comparisons, shifts;
36
37     cout << "=== Run 1: original array A ===" << endl;
38     cout << "Original: "; printArray(A, n);
39     insertionSort(A, n, comparisons, shifts);
40     cout << "Sorted: "; printArray(A, n);
41     cout << "Comparisons: " << comparisons << " Shifts: " << shifts <<
        endl;
42
43     int B[] = {5, 7, 14, 19, 23, 32, 34, 62};
44     cout << "\n=== Run 2: sorted copy (fewest shifts) ===" << endl;
45     cout << "Original: "; printArray(B, n);
46     insertionSort(B, n, comparisons, shifts);
47     cout << "Sorted: "; printArray(B, n);
48     cout << "Comparisons: " << comparisons << " Shifts: " << shifts <<
        endl;
49
50     int C[] = {62, 34, 32, 23, 19, 14, 7, 5};
51     cout << "\n=== Run 3: reversed copy (most shifts) ===" << endl;
52     cout << "Original: "; printArray(C, n);
53     insertionSort(C, n, comparisons, shifts);
54     cout << "Sorted: "; printArray(C, n);
55     cout << "Comparisons: " << comparisons << " Shifts: " << shifts <<
        endl;
56     return 0;
57 }

```

8. Execution Snapshot



```
rath@RATH666:~/eece7205/assignment01$ ./p2
i = 7, key = 19: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 20 Shifts: 15

=== Run 2: sorted copy (fewest shifts) ===
Original: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 7 Shifts: 0

=== Run 3: reversed copy (most shifts) ===
Original: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Shifts: 28
rath@RATH666:~/eece7205/assignment01$
```

Figure 2: Program output for Problem 2. Run 1 sorts the original array A, Run 2 sorts an already sorted copy, Run 3 sorts a reversed copy.

9. Answer to Question 4

Input	Comparisons	Shifts
A (random)	21	16
Sorted copy	7	0
Reversed copy	28	28

Fewest shifts: sorted input, every key needs only 1 comparison and 0 shifts, total $n - 1$ comparisons. Most shifts: reversed input, the i -th key needs to compare with all i sorted elements, total $n(n - 1)/2 = 28$ comparisons and 28 shifts. Shifts equal the number of inversions. Insertion Sort is very efficient on almost-sorted arrays.

Problem 3: Selection Sort

1. Understanding the Problem

Goal: sort A using Selection Sort. Output: each pass prints the minimum index and value, the array after the swap, and total comparisons and swaps.

2. Solution Approach

Selection Sort divides the array into a sorted prefix and an unsorted suffix. Pass i finds the smallest element in $\text{arr}[i..n-1]$, and if it is not already at position i , swaps it with $\text{arr}[i]$. At most $n - 1$

swaps.

3. Pseudocode

```
1 PROCEDURE printArray(arr, n)
2 1. FOR k <- 0 TO n - 1 DO PRINT arr[k] followed by space
3 2. PRINT newline
4
5 PROCEDURE selectionSort(arr, n, comparisons, swaps)
6 1. comparisons <- 0; swaps <- 0
7 2. FOR i <- 0 TO n - 2 DO
8 3.   minIdx <- i
9 4.   FOR j <- i + 1 TO n - 1 DO
10 5.     comparisons <- comparisons + 1
11 6.     IF arr[j] < arr[minIdx] THEN minIdx <- j
12 7.   END FOR
13 8.   PRINT "Pass", i + 1, ": min =", arr[minIdx],
14     "at index", minIdx, "->"
15 9.   IF minIdx != i THEN
16 10.     SWAP arr[i] AND arr[minIdx]
17 11.     swaps <- swaps + 1
18 12.   END IF
19 13.   CALL printArray(arr, n)
20 14. END FOR
```

4. Hand Trace

Pass	min value	min index	Swapped?	Array after the pass
1	5	4	yes	5 7 23 32 34 62 14 19
2	7	1	no	5 7 23 32 34 62 14 19
3	14	6	yes	5 7 14 32 34 62 23 19
4	19	7	yes	5 7 14 19 34 62 23 32
5	23	6	yes	5 7 14 19 23 62 34 32
6	32	7	yes	5 7 14 19 23 32 34 62
7	34	6	no	5 7 14 19 23 32 34 62
Total			5 swaps	28 comparisons

5. Correctness

- Invariant: before pass i , $\text{arr}[0..i-1]$ holds the i smallest elements in sorted order.
- Termination: after $n - 1$ passes all elements are in place.
- Duplicates: comparisons use $<$, so first occurrence wins, but swaps can change relative order. Selection Sort is **not stable**.
- $n = 0$ or $n = 1$: outer loop does not run.

6. Complexity Analysis

Case	Comparisons	Swaps	Time
Best	$n(n-1)/2$	0	$O(n^2)$
Average	$n(n-1)/2$	up to $n-1$	$O(n^2)$
Worst	$n(n-1)/2$	up to $n-1$	$O(n^2)$

7. C++ Program

```
1 // Assignment01_Problem03_Ge.cpp
2 #include <iostream>
3 using namespace std;
4
5 void printArray(const int arr[], int n) {
6     for (int k = 0; k < n; k++) {
7         cout << arr[k];
8         if (k < n - 1) cout << " ";
9     }
10    cout << endl;
11 }
12
13 void selectionSort(int arr[], int n, int &comparisons, int &swaps) {
14     comparisons = 0; swaps = 0;
15     for (int i = 0; i < n - 1; i++) {
16         int minIdx = i;
17         for (int j = i + 1; j < n; j++) {
18             comparisons++;
19             if (arr[j] < arr[minIdx]) minIdx = j;
20         }
21         cout << "Pass " << i + 1 << ": min = " << arr[minIdx]
22              << " at index " << minIdx << " -> ";
23         if (minIdx != i) {
24             int temp = arr[i];
25             arr[i] = arr[minIdx];
26             arr[minIdx] = temp;
27             swaps++;
28         }
29         printArray(arr, n);
30     }
31 }
32
33 int main() {
34     int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
35     int n = sizeof(A) / sizeof(A[0]);
36     int comparisons, swaps;
37
38     cout << "Original: "; printArray(A, n);
39     selectionSort(A, n, comparisons, swaps);
40     cout << "Sorted: "; printArray(A, n);
41     cout << "Comparisons: " << comparisons << " Swaps: " << swaps << endl;
42     return 0;
}
```

8. Execution Snapshot

```

rath@RATH666:~/eece7205/assignment01$ ./p3
Original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Swaps: 5
rath@RATH666:~/eece7205/assignment01$

```

Figure 3: Program output for Problem 3.

9. Answer to Question 4

Algorithm	Comparisons	Swaps
Bubble Sort	25	15
Selection Sort	28	5

Selection Sort makes far fewer swaps. Bubble Sort swaps every time it finds an inversion, and A has 15 inversions. Selection Sort swaps at most once per pass, so at most $n - 1 = 7$ swaps, and it actually made 5. Selection Sort compares more (28 vs 25) but swaps much less. When swap cost is high, Selection Sort is better.

Problem 4: Quick Sort (Lomuto Partition)

1. Understanding the Problem

Goal: sort A using Quick Sort with the last element as pivot (Lomuto partition). Output: i and j values for the first partition, array after every swap, pivot final position, subarray for every recursive call, and total comparisons.

2. Solution Approach

Quick Sort picks a pivot, partitions the array so that smaller elements are on the pivot's left and larger on the right, and recursively sorts both sides. Lomuto partition uses index i as the boundary of the "less than pivot" region, and j scans $\text{low}.. \text{high}-1$: if $\text{arr}[j] \leq \text{pivot}$, increment i and swap $\text{arr}[i]$ and $\text{arr}[j]$. Finally swap $\text{arr}[i+1]$ with $\text{arr}[\text{high}]$ and return $i + 1$.

3. Pseudocode

```

1 PROCEDURE printArray(arr, n)
2   1. FOR k <- 0 TO n - 1 DO PRINT arr[k] followed by space
3   2. PRINT newline
4

```

```

5 PROCEDURE partition(arr, low, high, comparisons, depth)
6 1. pivot <- arr[high]
7 2. i <- low - 1
8 3. FOR j <- low TO high - 1 DO
9 4.     comparisons <- comparisons + 1
10 5.     IF arr[j] <= pivot THEN
11 6.         i <- i + 1
12 7.         SWAP arr[i] AND arr[j]
13 8.         PRINT indent(depth), "swap i =", i, ", j =", j, ": "
14 9.         CALL printArray(arr + low, high - low + 1)
15 10.    END IF
16 11. END FOR
17 12. SWAP arr[i + 1] AND arr[high]
18 13. PRINT indent(depth), "pivot =", pivot, " placed at index", i + 1
19 14. RETURN i + 1
20
21 PROCEDURE quickSort(arr, low, high, comparisons, depth)
22 1. IF low < high THEN
23 2.     PRINT indent(depth), "Sorting subarray: "
24 3.     CALL printArray(arr + low, high - low + 1)
25 4.     p <- partition(arr, low, high, comparisons, depth)
26 5.     quickSort(arr, low, p - 1, comparisons, depth + 1)
27 6.     quickSort(arr, p + 1, high, comparisons, depth + 1)
28 7. END IF

```

4. Hand Trace

Depth	Subarray	Pivot	Pivot index	Left	Right
0	34 7 23 32 5 62 14 19	19	3	7 5 14	34 62 23 32
1	7 5 14	14	2	7 5	—
1	34 62 23 32	32	5	23	34 62
2	7 5	5	0	—	7
2	34 62	62	7	34	—
Total				21 comparisons	

5. Correctness

- Partition invariant: `arr[low..i]` are all $\leq$ pivot, `arr[i+1..j-1]` are all $>$ pivot.
- Termination: each partition puts the pivot in its final place, subproblems strictly shrink.
- Duplicates: Lomuto uses `<=`, equal elements may be swapped. Quick Sort is **not stable**.
- $n = 0$ or $n = 1$: `low < high` is false, return immediately.

6. Complexity Analysis

Case	Input	Comparisons	Time
Best	even split	about $n \log n$	$O(n \log n)$
Average	random	about $1.39 n \log n$	$O(n \log n)$
Worst	sorted / reversed	$n(n-1)/2$	$O(n^2)$

7. C++ Program

```
1 // Assignment01_Problem04_Ge.cpp
2 #include <iostream>
3 using namespace std;
4
5 void printArray(const int arr[], int n) {
6     for (int k = 0; k < n; k++) {
7         cout << arr[k];
8         if (k < n - 1) cout << " ";
9     }
10    cout << endl;
11 }
12
13 void printIndent(int depth) {
14     for (int d = 0; d < depth; d++) cout << " ";
15 }
16
17 int partition(int arr[], int low, int high, int &comparisons, int depth) {
18     int pivot = arr[high];
19     int i = low - 1;
20     for (int j = low; j < high; j++) {
21         comparisons++;
22         if (arr[j] <= pivot) {
23             i++;
24             int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
25             printIndent(depth);
26             cout << "swap i = " << i << ", j = " << j << ": ";
27             printArray(arr + low, high - low + 1);
28         }
29     }
30     int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
31     printIndent(depth);
32     cout << "pivot = " << pivot << " placed at index " << (i + 1) << endl;
33     return i + 1;
34 }
35
36 void quickSort(int arr[], int low, int high, int &comparisons, int depth)
37 {
38     if (low < high) {
39         printIndent(depth);
40         cout << "Sorting subarray: ";
41         printArray(arr + low, high - low + 1);
42         int p = partition(arr, low, high, comparisons, depth);
```

```

42     quickSort(arr, low, p - 1, comparisons, depth + 1);
43     quickSort(arr, p + 1, high, comparisons, depth + 1);
44 }
45 }
46
47 int main() {
48     int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
49     int n = sizeof(A) / sizeof(A[0]);
50     int comparisons = 0;
51
52     cout << "=== Run 1: original array A ===" << endl;
53     cout << "Original: "; printArray(A, n);
54     quickSort(A, 0, n - 1, comparisons, 0);
55     cout << "Sorted: "; printArray(A, n);
56     cout << "Comparisons: " << comparisons << endl;
57
58     int B[] = {5, 7, 14, 19, 23, 32, 34, 62};
59     comparisons = 0;
60     cout << "\n=== Run 2: sorted copy (worst case) ===" << endl;
61     cout << "Original: "; printArray(B, n);
62     quickSort(B, 0, n - 1, comparisons, 0);
63     cout << "Sorted: "; printArray(B, n);
64     cout << "Comparisons: " << comparisons << endl;
65     return 0;
66 }

```

8. Execution Snapshot

```

● rath@RATH666:~/eece7205/assignment01$ ./p4
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
Sorting subarray: 34 7 23 32 5 62 14 19
swap i = 0, j = 1: 7 34 23 32 5 62 14 19
swap i = 1, j = 4: 7 5 23 32 34 62 14 19
swap i = 2, j = 6: 7 5 14 32 34 62 23 19
pivot = 19 placed at index 3
Sorting subarray: 7 5 14
swap i = 0, j = 0: 7 5 14
swap i = 1, j = 1: 7 5 14
pivot = 14 placed at index 2
Sorting subarray: 7 5
pivot = 5 placed at index 0
Sorting subarray: 34 62 23 32
swap i = 4, j = 6: 23 62 34 32
pivot = 32 placed at index 5
Sorting subarray: 34 62
swap i = 6, j = 6: 34 62
pivot = 62 placed at index 7
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 14

```

Figure 4: Program output for Problem 4, Run 1 (original array A).

```

=== Run 2: sorted copy (worst case) ===
Original: 5 7 14 19 23 32 34 62
Sorting subarray: 5 7 14 19 23 32 34 62
swap i = 0, j = 0: 5 7 14 19 23 32 34 62
swap i = 1, j = 1: 5 7 14 19 23 32 34 62
swap i = 2, j = 2: 5 7 14 19 23 32 34 62
swap i = 3, j = 3: 5 7 14 19 23 32 34 62
swap i = 4, j = 4: 5 7 14 19 23 32 34 62
swap i = 5, j = 5: 5 7 14 19 23 32 34 62
swap i = 6, j = 6: 5 7 14 19 23 32 34 62
pivot = 62 placed at index 7
Sorting subarray: 5 7 14 19 23 32 34
swap i = 0, j = 0: 5 7 14 19 23 32 34
swap i = 1, j = 1: 5 7 14 19 23 32 34
swap i = 2, j = 2: 5 7 14 19 23 32 34
swap i = 3, j = 3: 5 7 14 19 23 32 34
swap i = 4, j = 4: 5 7 14 19 23 32 34
swap i = 5, j = 5: 5 7 14 19 23 32 34
pivot = 34 placed at index 6
Sorting subarray: 5 7 14 19 23 32
swap i = 0, j = 0: 5 7 14 19 23 32
swap i = 1, j = 1: 5 7 14 19 23 32
swap i = 2, j = 2: 5 7 14 19 23 32
swap i = 3, j = 3: 5 7 14 19 23 32
swap i = 4, j = 4: 5 7 14 19 23 32
pivot = 32 placed at index 5
Sorting subarray: 5 7 14 19 23
swap i = 0, j = 0: 5 7 14 19 23
swap i = 1, j = 1: 5 7 14 19 23
swap i = 2, j = 2: 5 7 14 19 23
swap i = 3, j = 3: 5 7 14 19 23
pivot = 23 placed at index 4
Sorting subarray: 5 7 14 19
swap i = 0, j = 0: 5 7 14 19
swap i = 1, j = 1: 5 7 14 19
swap i = 2, j = 2: 5 7 14 19
pivot = 19 placed at index 3
Sorting subarray: 5 7 14
swap i = 0, j = 0: 5 7 14
swap i = 1, j = 1: 5 7 14
pivot = 14 placed at index 2
Sorting subarray: 5 7
swap i = 0, j = 0: 5 7
pivot = 7 placed at index 1
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28

```

Figure 5: Program output for Problem 4, Run 2 (sorted copy, worst case).

9. Answer to Question 5

Input	Comparisons
Original A	21
Sorted copy	28

Why sorted is the worst case: with the last element as pivot, if the array is sorted, the pivot is the maximum. After partition, the pivot is at the rightmost position, the left subarray has size $n - 1$, the right is empty. Recursion depth is n , and each partition compares $n - 1, n - 2, \dots, 1$ elements, total $n(n - 1)/2 = 28$, degrading to $O(n^2)$.

How to avoid: (1) random pivot: pick a random element, swap it with the last, then partition. (2) median-of-three: pick the median of low, mid, high. (3) use Hoare partition.

Problem 5: Merge Sort

1. Understanding the Problem

Goal: sort A using Merge Sort, with $\text{mid} = (\text{low} + \text{high})/2$. Output: every split, every merge, total comparisons, and comparisons in the final merge.

2. Solution Approach

Merge Sort recursively divides the array into halves, sorts each half, then merges the two sorted halves. The merge uses a temporary array. Needs $O(n)$ extra memory.

3. Pseudocode

```
1  PROCEDURE printArray(arr, n)
2  1. FOR k <- 0 TO n - 1 DO PRINT arr[k] followed by space
3  2. PRINT newline
4
5  PROCEDURE merge(arr, low, mid, high, comparisons, finalMergeComparisons)
6  1. n1 <- mid - low + 1; n2 <- high - mid
7  2. LEFT <- new array of size n1
8  3. RIGHT <- new array of size n2
9  4. FOR i <- 0 TO n1 - 1 DO LEFT[i] <- arr[low + i]
10 5. FOR j <- 0 TO n2 - 1 DO RIGHT[j] <- arr[mid + 1 + j]
11 6. i <- 0; j <- 0; k <- low
12 7. WHILE i < n1 AND j < n2 DO
13 8.     comparisons <- comparisons + 1
14 9.     IF LEFT[i] <= RIGHT[j] THEN arr[k++] <- LEFT[i++]
15 10.    ELSE arr[k++] <- RIGHT[j++]
16 11. END WHILE
17 12. WHILE i < n1 DO arr[k++] <- LEFT[i++]
18 13. WHILE j < n2 DO arr[k++] <- RIGHT[j++]
19 14. PRINT "merged: "; CALL printArray(arr + low, high - low + 1)
20 15. DELETE LEFT, RIGHT
21
22 PROCEDURE mergeSort(arr, low, high, comparisons, finalMergeComparisons)
23 1. IF low < high THEN
24 2.     PRINT "split: "; CALL printArray(arr + low, high - low + 1)
25 3.     mid <- (low + high) / 2
26 4.     mergeSort(arr, low, mid, comparisons, finalMergeComparisons)
27 5.     mergeSort(arr, mid + 1, high, comparisons, finalMergeComparisons)
28 6.     merge(arr, low, mid, high, comparisons, finalMergeComparisons)
29 7. END IF
```

4. Hand Trace on

Merge step	Comparisons	Array after merge
7 34	1	7 34
23 32	1	23 32
7 23 32 34	2	7 23 32 34
5 62	1	5 62
14 19	1	14 19
5 14 19 62	3	5 14 19 62
Final merge	6	5 7 14 19 23 32 34 62
Total	15	

5. Correctness

- Recursive invariant: `mergeSort(arr, low, high)` returns with `arr[low..high]` sorted.
- Merge correctness: both halves are sorted, merging always takes the smaller current element.
- Termination: subarray length strictly decreases, stop when `low >= high`.
- Duplicates: `LEFT[i] <= RIGHT[j]` keeps equal elements in left half first. Merge Sort is **stable**.

6. Complexity Analysis

Case	Comparisons	Time	Space
Best	about $(n \log n)/2$	$O(n \log n)$	$O(n)$
Average	about $n \log n - n + 1$	$O(n \log n)$	$O(n)$
Worst	about $n \log n - n + 1$	$O(n \log n)$	$O(n)$

7. C++ Program

```
1 // Assignment01_Problem05_Ge.cpp
2 #include <iostream>
3 using namespace std;
4
5 void printArray(const int arr[], int n) {
6     for (int k = 0; k < n; k++) {
7         cout << arr[k];
8         if (k < n - 1) cout << " ";
9     }
10    cout << endl;
11 }
12
13 void merge(int arr[], int low, int mid, int high, int &comparisons, int &
    finalMerge) {
14     int n1 = mid - low + 1;
15     int n2 = high - mid;
```

```

16     int *L = new int[n1];
17     int *R = new int[n2];
18     for (int i = 0; i < n1; i++) L[i] = arr[low + i];
19     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
20
21     int i = 0, j = 0, k = low;
22     int before = comparisons;
23     while (i < n1 && j < n2) {
24         comparisons++;
25         if (L[i] <= R[j]) arr[k++] = L[i++];
26         else arr[k++] = R[j++];
27     }
28     while (i < n1) arr[k++] = L[i++];
29     while (j < n2) arr[k++] = R[j++];
30
31     if (low == 0 && high == 7) finalMerge = comparisons - before;
32
33     cout << "merged: ";
34     printArray(arr + low, high - low + 1);
35     delete[] L; delete[] R;
36 }
37
38 void mergeSort(int arr[], int low, int high, int &comparisons, int &
    finalMerge) {
39     if (low < high) {
40         cout << "split: ";
41         printArray(arr + low, high - low + 1);
42         int mid = (low + high) / 2;
43         mergeSort(arr, low, mid, comparisons, finalMerge);
44         mergeSort(arr, mid + 1, high, comparisons, finalMerge);
45         merge(arr, low, mid, high, comparisons, finalMerge);
46     }
47 }
48
49 int main() {
50     int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
51     int n = sizeof(A) / sizeof(A[0]);
52     int comparisons = 0, finalMerge = 0;
53
54     cout << "=== Run 1: original array A ===" << endl;
55     cout << "Original: "; printArray(A, n);
56     mergeSort(A, 0, n - 1, comparisons, finalMerge);
57     cout << "Sorted: "; printArray(A, n);
58     cout << "Comparisons: " << comparisons
59         << " (final merge: " << finalMerge << ")" << endl;
60
61     int B[] = {5, 7, 14, 19, 23, 32, 34, 62};
62     comparisons = 0; finalMerge = 0;
63     cout << "\n=== Run 2: sorted copy ===" << endl;
64     cout << "Original: "; printArray(B, n);
65     mergeSort(B, 0, n - 1, comparisons, finalMerge);
66     cout << "Sorted: "; printArray(B, n);
67     cout << "Comparisons: " << comparisons
68         << " (final merge: " << finalMerge << ")" << endl;

```

```

69
70     int C[] = {62, 34, 32, 23, 19, 14, 7, 5};
71     comparisons = 0; finalMerge = 0;
72     cout << "\n=== Run 3: reversed copy ===" << endl;
73     cout << "Original: "; printArray(C, n);
74     mergeSort(C, 0, n - 1, comparisons, finalMerge);
75     cout << "Sorted: "; printArray(C, n);
76     cout << "Comparisons: " << comparisons
77         << " (final merge: " << finalMerge << ")" << endl;
78     return 0;
79 }

```

8. Execution Snapshot

```

● rath@RATH666:~/eece7205/assignment01$ ./p5
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 17 (final merge: 7)

=== Run 2: sorted copy ===
Original: 5 7 14 19 23 32 34 62
split: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
merged: 5 7
split: 14 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
merged: 23 32
split: 34 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

```

Figure 6: Program output for Problem 5, Run 1 (original array A).

```

=== Run 3: reversed copy ===
Original: 62 34 32 23 19 14 7 5
split: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
merged: 34 62
split: 32 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
merged: 14 19
split: 7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12 (final merge: 4)

```

Figure 7: Program output for Problem 5, Run 2 and Run 3 (sorted and reversed copies).

9. Answer to Question 5

Input	Comparisons	Final merge comparisons
Original A	15	6
Sorted copy	11	4
Reversed copy	12	4

Merge Sort always splits the array in half, so recursion depth is $\log n$, and each merge level compares at most n elements. Total comparisons are always around $n \log n$, with little variation. Bubble Sort ranges from $O(n)$ to $O(n^2)$, Quick Sort from $O(n \log n)$ to $O(n^2)$. Merge Sort needs $O(n)$ extra memory for the temporary arrays, which is its main drawback.

Problem 6: Heap Sort

1. Understanding the Problem

Goal: sort A using Heap Sort. Output: heap after build-max-heap, heap part and sorted part after each sorting pass, total swaps inside `siftDown`.

2. Solution Approach

Heap Sort treats the array as a binary max-heap. Index k 's children are at $2k + 1$ and $2k + 2$. Phase 1: from the last non-leaf $n/2 - 1$ down to 0, call `siftDown` to build a max-heap. Phase 2: from `end = n - 1` down to 1, swap `arr[0]` with `arr[end]`, shrink the heap, and sift down the root.

3. Pseudocode

```

1 PROCEDURE printArray(arr, n)
2   1. FOR k <- 0 TO n - 1 DO PRINT arr[k] followed by space
3   2. PRINT newline
4
5 PROCEDURE siftDown(arr, root, size, swaps)
6   1. WHILE TRUE DO
7     2.     largest <- root

```

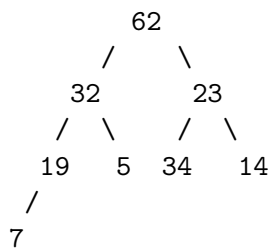
```

8  3.    left <- 2 * root + 1
9  4.    right <- 2 * root + 2
10 5.    IF left < size AND arr[left] > arr[largest] THEN largest <- left
11 6.    IF right < size AND arr[right] > arr[largest] THEN largest <- right
12 7.    IF largest = root THEN BREAK
13 8.    SWAP arr[root] AND arr[largest]
14 9.    swaps <- swaps + 1
15 10.   root <- largest
16 11. END WHILE
17
18 PROCEDURE heapSort(arr, n, swaps)
19 1. swaps <- 0
20 2. FOR i <- n / 2 - 1 DOWN TO 0 DO
21 3.    CALL siftDown(arr, i, n, swaps)
22 4. END FOR
23 5. PRINT "Heap: "; CALL printArray(arr, n)
24 6. FOR end <- n - 1 DOWN TO 1 DO
25 7.    SWAP arr[0] AND arr[end]; swaps <- swaps + 1
26 8.    CALL siftDown(arr, 0, end, swaps)
27 9.    PRINT "end =", end, " heap: "; CALL printArray(arr, end)
28 10.   PRINT "          sorted: "; CALL printArray(arr + end, n - end)
29 11. END FOR

```

4. Hand Trace

Max-heap after build phase:



Sorting phase summary:

end	heap after siftDown	sorted part
7	34 32 23 19 5 7 14	62
6	32 19 23 14 5 7	34 62
5	23 19 7 14 5	32 34 62
4	19 14 7 5	23 32 34 62
3	14 5 7	19 23 32 34 62
2	7 5	14 19 23 32 34 62
1	5	7 14 19 23 32 34 62

5. Correctness

- Heap invariant: after `siftDown`, the subtree rooted at `root` satisfies the max-heap property.

- Build phase: from the last non-leaf backwards, the whole array becomes a max-heap.
- Sort phase: each step moves the max (root) to `arr[end]` and restores the heap.
- Termination: build phase finite; sort phase `end` from $n - 1$ down to 1.
- Duplicates: `siftDown` uses `>`, equal values are not swapped, but the root swap can change relative order. Heap Sort is **not stable**.

6. Complexity Analysis

Case	Comparisons	Time
Best	$O(n \log n)$	$O(n \log n)$
Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n \log n)$	$O(n \log n)$

7. C++ Program

```

1 // Assignment01_Problem06_Ge.cpp
2 #include <iostream>
3 using namespace std;
4
5 void printArray(const int arr[], int n) {
6     for (int k = 0; k < n; k++) {
7         cout << arr[k];
8         if (k < n - 1) cout << " ";
9     }
10    cout << endl;
11 }
12
13 void siftDown(int arr[], int root, int size, int &swaps) {
14     while (true) {
15         int largest = root;
16         int left = 2 * root + 1;
17         int right = 2 * root + 2;
18         if (left < size && arr[left] > arr[largest]) largest = left;
19         if (right < size && arr[right] > arr[largest]) largest = right;
20         if (largest == root) break;
21         int temp = arr[root]; arr[root] = arr[largest]; arr[largest] =
            temp;
22         swaps++;
23         root = largest;
24     }
25 }
26
27 void heapSort(int arr[], int n, int &swaps) {
28     swaps = 0;
29     for (int i = n / 2 - 1; i >= 0; i--)
30         siftDown(arr, i, n, swaps);
31     cout << "Heap: "; printArray(arr, n);

```

```

32
33     for (int end = n - 1; end >= 1; end--) {
34         int temp = arr[0]; arr[0] = arr[end]; arr[end] = temp;
35         swaps++;
36         siftDown(arr, 0, end, swaps);
37         cout << "end = " << end << " heap: "; printArray(arr, end);
38         cout << "          sorted: "; printArray(arr + end, n - end);
39     }
40 }
41
42 int main() {
43     int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
44     int n = sizeof(A) / sizeof(A[0]);
45     int swaps;
46
47     cout << "Original: "; printArray(A, n);
48     heapSort(A, n, swaps);
49     cout << "Sorted: "; printArray(A, n);
50     cout << "Swaps in siftDown: " << swaps << endl;
51     return 0;
52 }

```

8. Execution Snapshot

```

● rath@RATH666:~/eece7205/assignment01$ ./p6
Original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7 heap: 34 32 23 19 5 7 14
      sorted: 62
end = 6 heap: 32 19 23 14 5 7
      sorted: 34 62
end = 5 heap: 23 19 7 14 5
      sorted: 32 34 62
end = 4 heap: 19 14 7 5
      sorted: 23 32 34 62
end = 3 heap: 14 5 7
      sorted: 19 23 32 34 62
end = 2 heap: 7 5
      sorted: 14 19 23 32 34 62
end = 1 heap: 5
      sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 19

```

Figure 8: Program output for Problem 6.

9. Answer to Question 5: Stability

Heap Sort is **not stable**. Counterexample: array $[5_a, 5_b, 3]$. Build max-heap: root 5_a , children $5_b, 3$. Sort phase $\text{end}=2$: swap 5_a with 3, array $[3, 5_b, 5_a]$; $\text{siftDown}(0,2)$: 3 swaps with 5_b , array $[5_b, 3, 5_a]$. $\text{end}=1$: swap 5_b with 3, array $[3, 5_b, 5_a]$. Final 5_b is before 5_a , reversing the original order. So Heap Sort is not stable.

Problem 7: Comparing Six Algorithms

1. Understanding the Problem

Write one program that includes the six sorting functions (with step printing off), makes a table of comparisons on three inputs, and measures running times on random arrays of 1000, 5000, 10000 integers. Use `<chrono>` and `<random>`.

2. Solution Approach

Reuse the six sorting functions from Problems 1–6, remove printing, add comparison counters. Generate one random array per size and copy it into six fresh arrays. Time each sort with `high_resolution_clock`.

3. Pseudocode

```
1  PROCEDURE printArray(arr, n)
2  1. FOR k <- 0 TO n - 1 DO PRINT arr[k] followed by space
3  2. PRINT newline
4
5  PROCEDURE bubbleSortCount(arr, n, comparisons)
6  PROCEDURE insertionSortCount(arr, n, comparisons)
7  PROCEDURE selectionSortCount(arr, n, comparisons)
8  PROCEDURE quickSortCount(arr, low, high, comparisons)
9  PROCEDURE mergeSortCount(arr, low, high, comparisons)
10 PROCEDURE heapSortCount(arr, n, comparisons)
11
12 MAIN
13 1. A <- {34, 7, 23, 32, 5, 62, 14, 19}
14 2. sortedA <- sorted A; reversedA <- reversed A
15 3. FOR each input IN {A, sortedA, reversedA} DO
16 4.     Run each algorithm, record comparisons
17 5. END FOR
18 6. FOR size IN {1000, 5000, 10000} DO
19 7.     base <- size random integers in 1..100000
20 8.     FOR each algorithm DO
21 9.         copy base to temp
22 10.        start <- high_resolution_clock::now()
23 11.        CALL algorithm(temp, size)
24 12.        end <- high_resolution_clock::now()
25 13.        record time
26 14.    END FOR
27 15. END FOR
28 16. For each algorithm, print first 10 elements of the sorted A
```

4. Comparisons on Fixed Inputs

Algorithm	A (original)	Sorted A	Reversed A
Bubble	25	7	28
Insertion	21	7	28
Selection	28	28	28
Quick	21	28	28
Merge	15	11	12
Heap	17	17	17

5. Correctness

All six functions reuse the implementations from Problems 1–6 with printing removed. Each sorted array is checked with `printArray` on the first 10 elements. Random data is copied fresh for each algorithm to make the comparison fair.

6. Complexity Analysis

Algorithm	Best	Average	Worst	Space
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

7. C++ Program

```
1 // Assignment01_Problem07_Ge.cpp
2 #include <iostream>
3 #include <chrono>
4 #include <random>
5 #include <cstring>
6 #include <string>
7 using namespace std;
8 using namespace std::chrono;
9
10 void printArray(const int arr[], int n) {
11     for (int k = 0; k < n; k++) {
12         cout << arr[k];
13         if (k < n - 1) cout << " ";
14     }
15     cout << endl;
16 }
17
18 void bubbleSortCount(int arr[], int n, long long &comparisons) {
19     comparisons = 0;
```

```

20     for (int i = 0; i < n - 1; i++) {
21         bool swapped = false;
22         for (int j = 0; j < n - 1 - i; j++) {
23             comparisons++;
24             if (arr[j] > arr[j + 1]) {
25                 int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
26                 swapped = true;
27             }
28         }
29         if (!swapped) break;
30     }
31 }
32
33 void insertionSortCount(int arr[], int n, long long &comparisons) {
34     comparisons = 0;
35     for (int i = 1; i < n; i++) {
36         int key = arr[i], j = i - 1;
37         while (j >= 0) {
38             comparisons++;
39             if (arr[j] > key) { arr[j + 1] = arr[j]; j--; }
40             else break;
41         }
42         arr[j + 1] = key;
43     }
44 }
45
46 void selectionSortCount(int arr[], int n, long long &comparisons) {
47     comparisons = 0;
48     for (int i = 0; i < n - 1; i++) {
49         int minIdx = i;
50         for (int j = i + 1; j < n; j++) {
51             comparisons++;
52             if (arr[j] < arr[minIdx]) minIdx = j;
53         }
54         if (minIdx != i) {
55             int t = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = t;
56         }
57     }
58 }
59
60 int partitionCount(int arr[], int low, int high, long long &comparisons) {
61     int pivot = arr[high], i = low - 1;
62     for (int j = low; j < high; j++) {
63         comparisons++;
64         if (arr[j] <= pivot) {
65             i++;
66             int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
67         }
68     }
69     int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;
70     return i + 1;
71 }
72

```

```

73 void quickSortCount(int arr[], int low, int high, long long &comparisons)
74 {
75     if (low < high) {
76         int p = partitionCount(arr, low, high, comparisons);
77         quickSortCount(arr, low, p - 1, comparisons);
78         quickSortCount(arr, p + 1, high, comparisons);
79     }
80 }
81 void mergeCount(int arr[], int low, int mid, int high, long long &
82     comparisons) {
83     int n1 = mid - low + 1, n2 = high - mid;
84     int *L = new int[n1], *R = new int[n2];
85     for (int i = 0; i < n1; i++) L[i] = arr[low + i];
86     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
87     int i = 0, j = 0, k = low;
88     while (i < n1 && j < n2) {
89         comparisons++;
90         if (L[i] <= R[j]) arr[k++] = L[i++];
91         else arr[k++] = R[j++];
92     }
93     while (i < n1) arr[k++] = L[i++];
94     while (j < n2) arr[k++] = R[j++];
95     delete[] L; delete[] R;
96 }
97 void mergeSortCount(int arr[], int low, int high, long long &comparisons)
98 {
99     if (low < high) {
100         int mid = (low + high) / 2;
101         mergeSortCount(arr, low, mid, comparisons);
102         mergeSortCount(arr, mid + 1, high, comparisons);
103         mergeCount(arr, low, mid, high, comparisons);
104     }
105 }
106 void siftDownCount(int arr[], int root, int size, long long &comparisons)
107 {
108     while (true) {
109         int largest = root, left = 2 * root + 1, right = 2 * root + 2;
110         if (left < size) { comparisons++; if (arr[left] > arr[largest])
111             largest = left; }
112         if (right < size) { comparisons++; if (arr[right] > arr[largest])
113             largest = right; }
114         if (largest == root) break;
115         int t = arr[root]; arr[root] = arr[largest]; arr[largest] = t;
116         root = largest;
117     }
118 }
119 void heapSortCount(int arr[], int n, long long &comparisons) {
120     comparisons = 0;
121     for (int i = n / 2 - 1; i >= 0; i--)
122         siftDownCount(arr, i, n, comparisons);

```

```

121     for (int end = n - 1; end >= 1; end--) {
122         int t = arr[0]; arr[0] = arr[end]; arr[end] = t;
123         siftDownCount(arr, 0, end, comparisons);
124     }
125 }
126
127 void compareAll(const int base[], int n, const string &label) {
128     int *temp = new int[n];
129     long long cmp;
130     double ms;
131
132     cout << "\n--- " << label << " ---" << endl;
133
134     memcpy(temp, base, n * sizeof(int));
135     auto t0 = high_resolution_clock::now();
136     bubbleSortCount(temp, n, cmp);
137     auto t1 = high_resolution_clock::now();
138     ms = duration_cast<microseconds>(t1 - t0).count() / 1000.0;
139     cout << "Bubble:      comparisons = " << cmp << ", time = " << ms << "
140           ms" << endl;
141
142     memcpy(temp, base, n * sizeof(int));
143     t0 = high_resolution_clock::now();
144     insertionSortCount(temp, n, cmp);
145     t1 = high_resolution_clock::now();
146     ms = duration_cast<microseconds>(t1 - t0).count() / 1000.0;
147     cout << "Insertion: comparisons = " << cmp << ", time = " << ms << "
148           ms" << endl;
149
150     memcpy(temp, base, n * sizeof(int));
151     t0 = high_resolution_clock::now();
152     selectionSortCount(temp, n, cmp);
153     t1 = high_resolution_clock::now();
154     ms = duration_cast<microseconds>(t1 - t0).count() / 1000.0;
155     cout << "Selection: comparisons = " << cmp << ", time = " << ms << "
156           ms" << endl;
157
158     memcpy(temp, base, n * sizeof(int));
159     cmp = 0;
160     t0 = high_resolution_clock::now();
161     quickSortCount(temp, 0, n - 1, cmp);
162     t1 = high_resolution_clock::now();
163     ms = duration_cast<microseconds>(t1 - t0).count() / 1000.0;
164     cout << "Quick:      comparisons = " << cmp << ", time = " << ms << "
165           ms" << endl;
166
167     memcpy(temp, base, n * sizeof(int));
168     cmp = 0;
169     t0 = high_resolution_clock::now();
170     mergeSortCount(temp, 0, n - 1, cmp);
171     t1 = high_resolution_clock::now();
172     ms = duration_cast<microseconds>(t1 - t0).count() / 1000.0;
173     cout << "Merge:      comparisons = " << cmp << ", time = " << ms << "
174           ms" << endl;

```

```

170
171     memcpy(temp, base, n * sizeof(int));
172     t0 = high_resolution_clock::now();
173     heapSortCount(temp, n, cmp);
174     t1 = high_resolution_clock::now();
175     ms = duration_cast<microseconds>(t1 - t0).count() / 1000.0;
176     cout << "Heap:      comparisons = " << cmp << ", time = " << ms << "
        ms" << endl;
177
178     delete[] temp;
179 }
180
181 int main() {
182     int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
183     int n = 8;
184     int sortedA[] = {5, 7, 14, 19, 23, 32, 34, 62};
185     int reversedA[] = {62, 34, 32, 23, 19, 14, 7, 5};
186
187     cout << "==== Part 1: Comparisons on fixed inputs =====" <<
        endl;
188     compareAll(A, n, "Original A");
189     compareAll(sortedA, n, "Sorted A");
190     compareAll(reversedA, n, "Reversed A");
191
192     cout << "\n==== Part 2: Timing on random arrays =====" <<
        endl;
193     random_device rd;
194     mt19937 gen(rd());
195     uniform_int_distribution<> dist(1, 100000);
196
197     int sizes[] = {1000, 5000, 10000};
198     for (int s : sizes) {
199         int *base = new int[s];
200         for (int i = 0; i < s; i++) base[i] = dist(gen);
201         cout << "\n### Array size = " << s << " ###" << endl;
202         compareAll(base, s, "Random size " + to_string(s));
203         delete[] base;
204     }
205
206     cout << "\n==== Part 3: Verify sorted first 10 elements
        =====" << endl;
207     int small[] = {34, 7, 23, 32, 5, 62, 14, 19};
208     int temp[8];
209     long long c = 0;
210
211     memcpy(temp, small, sizeof(small)); bubbleSortCount(temp, 8, c);
212     cout << "Bubble: "; printArray(temp, 8);
213     memcpy(temp, small, sizeof(small)); insertionSortCount(temp, 8, c);
214     cout << "Insertion: "; printArray(temp, 8);
215     memcpy(temp, small, sizeof(small)); selectionSortCount(temp, 8, c);
216     cout << "Selection: "; printArray(temp, 8);
217     memcpy(temp, small, sizeof(small)); c = 0; quickSortCount(temp, 0, 7,
        c);
218     cout << "Quick: "; printArray(temp, 8);

```

```

219     memcpy(temp, small, sizeof(small)); c = 0; mergeSortCount(temp, 0, 7,
220         c);
221     cout << "Merge:      "; printArray(temp, 8);
222     memcpy(temp, small, sizeof(small)); heapSortCount(temp, 8, c);
223     cout << "Heap:       "; printArray(temp, 8);
224
225     return 0;
226 }

```

8. Execution Snapshot

```

● rath@RATH666:~/eece7205/assignment01$ ./p7
===== Part 1: Comparisons on fixed inputs =====

--- Original A ---
Bubble:  comparisons = 25, time = 0 ms
Insertion: comparisons = 20, time = 0 ms
Selection: comparisons = 28, time = 0 ms
Quick:   comparisons = 14, time = 0 ms
Merge:   comparisons = 17, time = 0.002 ms
Heap:    comparisons = 26, time = 0 ms

--- Sorted A ---
Bubble:  comparisons = 7, time = 0 ms
Insertion: comparisons = 7, time = 0 ms
Selection: comparisons = 28, time = 0 ms
Quick:   comparisons = 28, time = 0 ms
Merge:   comparisons = 12, time = 0 ms
Heap:    comparisons = 27, time = 0 ms

--- Reversed A ---
Bubble:  comparisons = 28, time = 0 ms
Insertion: comparisons = 28, time = 0 ms
Selection: comparisons = 28, time = 0 ms
Quick:   comparisons = 28, time = 0 ms
Merge:   comparisons = 12, time = 0 ms
Heap:    comparisons = 24, time = 0 ms

```

Figure 9: Program output for Problem 7, Part 1 (comparisons on fixed inputs).

```

===== Part 2: Timing on random arrays =====

### Array size = 1000 ###

--- Random size 1000 ---
Bubble:  comparisons = 499200, time = 0.94 ms
Insertion: comparisons = 247316, time = 0.326 ms
Selection: comparisons = 499500, time = 0.462 ms
Quick:    comparisons = 10967, time = 0.049 ms
Merge:    comparisons = 8692, time = 0.093 ms
Heap:     comparisons = 16863, time = 0.072 ms

### Array size = 5000 ###

--- Random size 5000 ---
Bubble:  comparisons = 12491172, time = 22.331 ms
Insertion: comparisons = 6240822, time = 7.859 ms
Selection: comparisons = 12497500, time = 10.239 ms
Quick:    comparisons = 68276, time = 0.268 ms
Merge:    comparisons = 55272, time = 0.412 ms
Heap:     comparisons = 107695, time = 0.401 ms

### Array size = 10000 ###

--- Random size 10000 ---
Bubble:  comparisons = 49987374, time = 87.418 ms
Insertion: comparisons = 24824273, time = 29.769 ms
Selection: comparisons = 49995000, time = 40.572 ms
Quick:    comparisons = 158367, time = 0.575 ms
Merge:    comparisons = 120491, time = 0.849 ms
Heap:     comparisons = 235385, time = 0.876 ms

===== Part 3: Verify sorted first 10 elements =====
Bubble:  5 7 14 19 23 32 34 62
Insertion: 5 7 14 19 23 32 34 62
Selection: 5 7 14 19 23 32 34 62
Quick:    5 7 14 19 23 32 34 62
Merge:    5 7 14 19 23 32 34 62
Heap:     5 7 14 19 23 32 34 62

```

Figure 10: Program output for Problem 7, Part 2 and Part 3 (timing and verification).

9. Answer to Question 4

Which algorithms grow like n^2 : Bubble, Insertion, Selection. Their comparisons at $n = 1000$ are about 500,000; at $n = 5000$ about 12,500,000 (25 $\times$); at $n = 10000$ about 50,000,000 (4 $\times$ the $n = 5000$ value), proportional to n^2 .

Which grow like $n \log n$: Quick, Merge, Heap. Going from $n = 1000$ to $n = 10000$ (10 $\times$) increases comparisons by about 10–12 $\times$, matching $n \log n$.

For a large random array, choose **Quick Sort**: average $O(n \log n)$, small constant, in-place, cache-friendly. If the worst case worries you, use random pivot or median-of-three.

For an almost-sorted array, choose **Insertion Sort**: close to $O(n)$, far faster than the other algorithms. If the data is huge and almost sorted, Merge Sort is also safe (stable, $O(n \log n)$), but Insertion Sort is simpler and faster.